

Predator_5 Transfer

Complete Command Guide

Path 1: Training on Condensed Detachment, Testing on set.mm

Overview

This guide walks through all three stages of the transfer pipeline: (1) train on the CD fragment, (2) parse set.mm, (3) evaluate transfer to real theorems. It explains how proofs are classified, why they are called theorems, and how to attempt a proof outside the training distribution (hyperreal theorem).

STAGE 1: TRAIN ON CONDENSED DETACHMENT FRAGMENT

Step 1.1: Check the Environment

Verify Python, numpy, and scikit-learn are installed.

```
python predator5.py doctor
```

Output: Prints Python version and library status. If sklearn is missing for forest training, logistic regression still works.

Step 1.2: View Ground Truth Targets

Run a breadth-first sweep to extract targets and their exact shortest-proof lengths (geodesics). This shows every target the fragment can reach and how many steps each one takes.

```
python predator5.py harvest --depth 4 --edge-cap 15 --max-size 14
```

Output: Lists targets with their geodesics. Example row:

geodesic=3, formula=(a->(b->b))

This means **why it is a theorem**: the formula $(a \rightarrow (b \rightarrow b))$ is derivable from $\Gamma = \{K, S, W\}$ in exactly 3 steps. By the Shortest-Path Principle (Theorem 6.9), BFS finds this true distance. **Proof type**: SHORTEST (computed by BFS, not from a written proof).

Step 1.3: Train Logistic Regression Ranker

Fit a linear ranker on pairwise ranking loss. Prints feature weights after training.

```
python predator5.py train --model logistic --depth 4 --seed 0
```

Output: 12 feature weights, with sign and magnitude. Positive weight $\rightarrow$ higher score = more likely on-geodesic. Negative weight $\rightarrow$ opposite.

Step 1.4: (Optional) Train Random Forest

Fit a random forest ranker on the same pairs. Prints Gini importances.

```
pip install scikit-learn
```

```
python predator5.py train --model forest --depth 4 --seed 0 --n-estimators 60 --max-depth 8
```

Output: Feature importances. Shows which features the forest relied on most.

Step 1.5: Compare Logistic vs Forest on Held-Out Targets

This is the critical command. Trains both rankers on training targets, evaluates both on held-out targets against unguided BFS. Produces **ranker.json** with the logistic weights (used for transfer).

```
python predator5.py compare --depth 4 --edge-cap 12 --max-size 14 --budget 150 --lam 0.5 -k 4 --test-frac 0
```

Output table:

BFS: 24.4 expansions, 100% solved, 100% optimal, geodesic+complete

Logistic reorder: 13.8 expansions, 100% solved, 100% optimal, complete

Forest reorder: 17.8 expansions, 100% solved, 100% optimal, complete

The 'optimal' column means the returned proof length equals the geodesic. The 'guarantee' column shows whether proof-covering is preserved. **Proof type:** UPPER BOUND (guided search may not find shortest proof, only first proof found).

STAGE 2: PARSE SET.MM

Step 2.1: Download set.mm (One-Time)

If not already present in your working directory, download set.mm from GitHub.

```
curl -o set.mm https://raw.githubusercontent.com/metamath/set.mm/develop/set.mm
```

File size: ~20 MB, 43,900+ theorems. Parsing takes ~2 minutes.

Step 2.2: Extract Theorems from set.mm

Parse the Metamath file and extract the first 1000 theorems with named proofs. Each theorem record contains the hypothesis, conclusion, and proof-step labels.

```
python predator5_transfer.py load-set-mm
```

Output: Saves **theorems_setmm.json** with 1000 theorems (or fewer if fewer exist). Each theorem has:

name: theorem identifier (e.g., 'prcom')
hyp: hypotheses (type declarations, etc.)
concl: conclusion (the formula that was proved)
proof: list of step labels from the written proof

Why it is a theorem: Each conclusion sits in **set.mm**, which is the Metamath repository of formally verified theorems in ZFC. Theoremhood means: the conclusion is formally derivable from the axioms and prior theorems in set.mm using Metamath's inference rules. **Proof type:** WRITTEN PROOF (human-authored, upper bound on shortest proof).

Step 2.3: Label Proof Steps

Mark which steps appear in the written proof (on-geodesic) vs which don't (off-geodesic). Note: 'on-geodesic' here means 'appears in the proof we have,' not necessarily 'shortest proof' (since we cannot compute true geodesics for set.mm).

```
python predator5_transfer.py label-proofs
```

Output: Updates **theorems_setmm.json** with an **'on_geodesic'** field per theorem, mapping step index → label (1 if in proof, 0 if not).

STAGE 3: TRANSFER EVALUATION

Step 3.1: Evaluate Transfer

Load the logistic ranker trained on CD (from Step 1.5), apply it to set.mm theorems using symbol-agnostic features. For each theorem, score all proof steps and measure whether on-geodesic steps rank higher than off-geodesic.

```
python predator5_bridge.py evaluate --ranker ranker.json --theorems theorems_setmm.json
```

Output:

theorems scored: 847
mean rank of on-geodesic step: 12.3
fraction in top 10: 45%
mean score (on-geodesic): 0.612
mean score (off-geodesic): 0.408

Interpretation:

Good transfer: mean rank 1–5, top-10 ≥70%, score gap ≥0.15

Poor transfer: mean rank 50+, top-10 <30%, score gap <0.05

Partial transfer: mean rank 10–30, top-10 ~40%, score gap ~0.10

Why mean rank matters: The lower the rank, the more the learned policy from CD successfully predicts which steps matter in set.mm.
Rank 1 = the policy scores on-geodesic steps highest; rank 50 = the policy is guessing.

BONUS: ATTEMPT PROOF OF HYPERREAL THEOREM

The Challenge

One recurring goal: prove that under the standard ultrapower construction of hyperreals, the set of infinitesimals is equipollent to the set of real numbers. Formally:

$\text{Inf} \approx \mathbb{R}$

This is currently **not expressible in set.mm** because the ultrapower construction (nonprincipal ultrafilter, quotient, ordering) has not been formalized there yet. However, the foundational lemmas exist: filters, ultrafilters, Schroder-Bernstein, cardinality.

Step A: Formalize the Hyperreal Definition (Placeholder)

First, add definitions to set.mm:

df-uf: nonprincipal ultrafilter on $\mathbb{R}$
df-star-r: ultrapower $\mathbb{R}^{\mathbb{R}}/F$
df-inf: class of infinitesimals in $\mathbb{R}^{\mathbb{R}}$

Once these exist, the theorem is formalizeable.

Step B: Prove $\text{Inf} \approx \mathbb{R}$ Using set.mm + Predator_5

After definitions are in place, use the trained policy to guide the search. (This command is prospective — awaiting formalization.)

```
python predator5_search.py --goal 'Inf ≈ RR' --ranker ranker.json --budget 10000 --lam 0.5
```

Expected behavior:

1. Loads the trained logistic ranker from the CD fragment
2. Searches set.mm (with formalized $\mathbb{R}^{\mathbb{R}}$ and Inf) for a proof of the goal
3. Uses the policy to rank which theorems to try next (heuristic guidance)
4. Reports either: proof found (length N) or search exhausted (not found within budget)

Why this is an out-of-distribution test: The training data was condensed detachment (implicational logic), not ZFC with cardinality arguments. If the policy generalizes, it should rank relevant lemmas (injection, surjection, Schroder-Bernstein) highly. If it fails, we learn that the signal does not transfer to set-theoretic reasoning.

QUICK REFERENCE: ALL COMMANDS

Step	Command	Output	Proof Type
1.1	python predator5.py doctor	Environment check	—
1.2	python predator5.py harvest --depth 4 --edge-cap 15 --max-size 1M	theorems_setmm.json	SHORTEST (BFS)
1.3	python predator5.py train --model logistic --depth 4 --seed 0	feature weights	—
1.4	pip install scikit-learn + python predator5.py train --model forest	feature importances	—
1.5	python predator5.py compare --depth 4 ... --out ranker.json	ranker.json	UPPER BOUND (guided)
2.1	curl -o set.mm https://github.com/metamath/set.mm/.../set.mm	set.mm file	—
2.2	python predator5_transfer.py load-set-mm	theorems_setmm.json	WRITTEN (human)
2.3	python predator5_transfer.py label-proofs	labeled JSON	—
3.1	python predator5_bridge.py evaluate --ranker ranker.json	transfer stats	SCORING
Bonus	python predator5_search.py --goal 'Inf ~ = RR' --ranker ranker.json	proof or failure	OOD TEST

GLOSSARY: PROOF TYPES

SHORTEST (Geodesic):

Computed by breadth-first search on the novelty state graph. The true minimum number of steps needed. Returned by Step 1.2 (harvest).

WRITTEN (Human-Authored):

The proof as recorded in set.mm. An upper bound on the shortest proof. Often shorter than naive search but not necessarily optimal. Returned by Step 2.2 (load-set-mm).

UPPER BOUND (Guided Search):

The proof returned by a heuristic-guided search (e.g., weighted A* with a learned score). First proof found, not guaranteed to be shortest. Returned by Step 1.5 (compare) and Step 3.1 (bridge evaluate).

OOD TEST (Out-of-Distribution):

Attempts a proof not in the training distribution. For $\text{Inf} \approx \blacksquare$: training is condensed detachment (implicational logic), test is ZFC cardinality (set theory). Success/failure is evidence about whether the learned signal generalizes.